

TECHNICAL DOCUMENTATION of Anviksha 8 – Bit CPU

ANVIKSHA 8- BIT CENTRAL PROCESSING UNIT (CPU)

Documentation by THIYANAND M (Project Lead)

This project consists of both a software and hardware implementation. The software model was developed using Logisim Evolution; however, due to clock cycle simulation limitations within the software, visual output is restricted despite the CPU executing logic perfectly. The hardware implementation, built entirely from scratch in Verilog, operates flawlessly. This document will focus exclusively on the hardware architecture and its physical implementation.

Only Hardware Implementation and its explanation will goes below.

Note: **This CPU is built from scratch.**

1. System Overview

This project implements a fully custom 8-bit central processing unit (CPU) written in Verilog and deployed on the Xilinx Artix-7 (Basys 3) FPGA. Inspired by the SAP-1 (Simple As Possible) architecture, the CPU features a custom instruction set, a microcoded control unit, and a hardware-multiplexed program loader.

Key Features:

- Word Size: 8-bit data bus and 8-bit memory addressing.
- Memory: 256-byte internal SRAM.
- Clocking: "Smart Clock" architecture featuring asynchronous switching between high-speed program loading and human-visible execution cycles.
- Control Unit: 1024x17 Microcode ROM driving a 17-bit control word.
- I/O: 7-segment multiplexed display output and LED bus monitoring.

2. Hardware Architecture

The processor is built using a shared-bus architecture where a central 8-bit `main_bus` facilitates data transfer between discrete registers.

Core Components

- ALU (Arithmetic Logic Unit): 8-bit ripple-carry adder/subtractor with Zero (ZF) and Carry (CF) condition flags.
- Registers:
 - Register A (Accumulator): Primary operational register for ALU and I/O.
 - Register B: Secondary operand register for ALU operations.
 - Instruction Register (IR): 8-bit register. The upper 5 bits feed the instruction decoder; the lower 4 bits can be pushed to the bus as a memory address.
 - Memory Address Register (MAR): Holds the current address for SRAM reads/writes.
 - Output Register (OUT): Latches data for the 7-segment display.
- Program Counter (PC): 8-bit synchronous counter with jump (parallel load) capabilities.

3. Instruction Set Architecture (ISA)

The CPU uses a custom 5-bit opcode space. Instructions are fetched, decoded, and executed over a maximum of 8 T-states (clock cycles) driven by a 3-bit step counter.

Assembly	Opcode (5-bit)	Hex Base	Description
----------	----------------	----------	-------------

TECHNICAL DOCUMENTATION of Anviksha 8 – Bit CPU

LDA	0	@00	Load Accumulator with memory contents
ADD	10	@10	Add memory contents to Accumulator
STA	100	@20	Store Accumulator into memory
JMP	1000	@40	Unconditional jump to a memory address
OUT	11100	@E0	Push Accumulator to Output Display
HLT	11111	@F8	Halt CPU execution

4. Control Logic & Microcode

The Instruction Decoder translates the current Opcode, Step Counter, and ALU Flags into a 17-bit Control Word that dictates bus arbitration and register states.

Bit	Signal	Description	Bit	Signal	Description
16	OI	Output Register In	7	BO	Register B Out
15	JP	Jump (PC Load)	6	BI	Register B In
14	CO	Program Counter Out	5	SU	ALU Subtract Mode
13	CE	Count Enable (PC Increment)	4	EO	ALU Out Enable
12	IR	Instruction Register In	3	AO	Register A Out
11	II	Instruction Register Out (4-bit)	2	AI	Register A In
10	RR	RAM Read Enable	1	HL	Halt CPU Clock
9	RW	RAM Write Enable	0	FI	ALU Flags In
8	MI	Memory Address Register In			

5. Physical Interface & Operation (Basys 3)

The CPU is designed to be physically interacted with via the FPGA development board.

- SW1 (Mode Select):
 - UP (Program Mode): Disconnects the CPU and grants the hardware program_loader exclusive access to the memory bus. Programs are loaded from ROM into SRAM at high speed (~3 MHz).
 - DOWN (Execute Mode): Wakes the CPU and steps the clock at a human-visible frequency (~3 Hz) for hardware debugging.
- SW0 (Master Reset): Active-high absolute reset. Clears the PC, all registers, and un-halts the CPU.
- Green LEDs: Dynamically switches to monitor the main_bus during Program Mode, and the out_reg during Execute Mode.

TECHNICAL DOCUMENTATION of Anviksha 8 – Bit CPU

- 7-Segment Display: Hardware multiplexed display showing the decimal/hexadecimal value of the Output Register.

6. Performance & Timing Analysis

The architecture features highly efficient, non-pipelined combinatorial logic. Post-implementation timing analysis in Vivado based on a standard 100 MHz clock (10 ns period) yielded the following metrics:

Worst Negative Slack (WNS): 7.297ns

- Theoretical Maximum Clock Frequency (F_{max}):

$$F_{max} = \frac{1}{10.000 \text{ ns} - 7.297 \text{ ns}} \approx 369.96 \text{ MHz}$$

- This indicates the silicon paths easily resolve within ~2.7 ns, allowing the processor to theoretically operate at up to ~370 MHz on the Artix-7 fabric.

7. Benchmark Software

To validate continuous looping, memory write-back, and ALU stability, the processor was benchmarked using the Fibonacci Sequence.

Memory Organization: Variables are loaded into hardware addresses 00 through 02. The executable code sits at 03 to 0B.

Fibonacci Assembly Code (rom_data.mem):

```
00: DATA 00 ; Variable 1 (Fn-2)
01: DATA 01 ; Variable 2 (Fn-1)
02: DATA 00 ; Variable 3 (Temp)
03: LDA 00 ; Load Fn-2 into Accumulator
04: OUT ; Display Sequence Number
05: ADD 01 ; Accumulator = Fn-2 + Fn-1 (Calculates Fn)
06: STA 02 ; Store Fn into Temp
07: LDA 01 ; Load Fn-1 into Accumulator
08: STA 00 ; Overwrite Fn-2 with Fn-1
09: LDA 02 ; Load Temp into Accumulator
0A: STA 01 ; Overwrite Fn-1 with Temp
0B: JMP 03 ; Infinite Loop
```

8. Simulation & Verification Methodology

To validate the CPU architecture before physical hardware synthesis, a comprehensive behavioral simulation environment was established in Xilinx Vivado. The simulation methodology was designed to perfectly mirror the physical interaction with the Basys 3 board while optimizing for computational time.

8.1 Environment-Aware Clocking

Simulating a 100 MHz clock up to 2^{24} divider ticks for human-visible speeds requires excessive simulation runtime. To solve this, the clock divider logic utilizes Vivado's compiler directives (`#ifndef SYNTHESIS`) to create an environment-aware clock:

- **During Simulation:** The CPU and loader automatically switch to high-speed clock taps (e.g., `clk_div[1]` and `clk_div[2]`), allowing full program loading and execution to simulate in under 100 microseconds.

TECHNICAL DOCUMENTATION of Anviksha 8 – Bit CPU

- **During Hardware Synthesis:** The compiler defaults back to the slower clk_div[22] and clk_div[24] taps for human-visible LED debugging.

8.2 Testbench Architecture

The master testbench instantiates the cpu_8bit_top module and generates a continuous 100 MHz master clock (10 ns period). It algorithmically mimics human interaction with the physical switches to transition the CPU through its primary states:

1. **Initialization:** Forces reset_sw (SW0) and prog_mode_sw (SW1) HIGH.
2. **Program Loading Phase:** Lowers reset_sw and holds prog_mode_sw HIGH for 15us, granting the hardware loader time to write all 256 addresses of the rom_data.mem file into the SRAM.
3. **Execution Phase:** Toggles reset_sw to zero out the Program Counter, then pulls prog_mode_sw LOW, transferring bus control back to the CPU to begin fetching instructions.

8.3 Waveform Analysis & Validation

Verification was confirmed via Vivado's waveform viewer by monitoring the internal signal pathways during the Fibonacci benchmark execution.

- **Bus Integrity:** The main_bus[7:0] was observed accurately transitioning between Instruction Fetch (from SRAM) and ALU execution output without high-Z collisions.
- **Microcode Accuracy:** The 17-bit ctrl_word was verified to perfectly match the expected binary states for LDA, ADD, STA, OUT, and JMP sequentially.
- **Mathematical Verification:** The out_reg_val[7:0] output successfully stepped through the hexadecimal equivalents of the Fibonacci sequence (00, 01, 01, 02, 03, 05, 08, 0D, 15...) validating both the ripple-carry ALU logic and the memory read/write cycles.

Sample Code for running in this CPU

2. PROGRAM 1: ADDING TWO VARIABLES

RAM Addr	Binary Code	Assembly	Description
----------	-------------	----------	-------------

00	00000100	LDA 04	Load data at Address 04 into Accumulator
01	00010101	ADD 05	Add data at Address 05 to Accumulator
02	11100000	OUT	Output Accumulator to 7-segment display
03	11111000	HLT	Halt CPU execution
04	00000101	DATA: 5	Variable 1 (Decimal 5)
05	00000011	DATA: 3	Variable 2 (Decimal 3)

2. PROGRAM 2: FIBONACCI SEQUENCE ($F_n = F_{n-1} + F_{n-2}$)

RAM Addr	Binary Code	Assembly	Description
----------	-------------	----------	-------------

TECHNICAL DOCUMENTATION of Anviksha 8 – Bit CPU

00	00000000	DATA: 0	Variable 1 (Fn-2) - Initialized to 0
01	00000001	DATA: 1	Variable 2 (Fn-1) - Initialized to 1
02	00000000	DATA: 0	Temporary storage variable
03	00000000	LDA 00	Load Variable 1 into Accumulator
04	11100000	OUT	Output current Fibonacci number to display
05	00010001	ADD 01	Add Variable 2 to Accumulator (Calculates new Fn)
06	00100010	STA 02	Store the new math result into Temp variable
07	00000001	LDA 01	Load Variable 2 back into Accumulator
08	00100000	STA 00	Overwrite Variable 1 with Variable 2 (Shift)
09	00000010	LDA 02	Load Temp result back into Accumulator
0A	00100001	STA 01	Overwrite Variable 2 with Temp result (Shift)
0B	01000011	JMP 03	Jump back to Address 03 to loop infinitely

Running the Simulation in Basys 3 using Xilinx Vivado

1. The Pre-Flight Check (Constraints File)

Before Vivado can program the board, it needs to know exactly which physical pins on the Basys 3 correspond to the inputs and outputs in your `cpu_8bit_top.v` file.

Make sure you have a constraints file (usually named `Basys3_Master.xdc`) added to your project under Constraints in the Sources window. Inside that file, ensure these specific ports are uncommented and match your code exactly:

- Clock: `clk_100MHz` (Pin W5)
- Switches: `reset_sw` for SW0 (Pin V17) and `prog_mode_sw` for SW1 (Pin V16)
- LEDs: `leds[0]` to `leds[7]` (Pins U16 to V14)
- 7-Segment Display: `cathode[0]` to `cathode[6]` (Pins W7 to U7) and `anode[0]` to `anode[3]` (Pins U2 to W4)

2. Generate the Bitstream

Now we tell Vivado to translate your Verilog code into the physical logic gate pathways for the Artix-7 chip.

1. On the far-left side of Vivado, look under the PROGRAM AND DEBUG section.
2. Click Generate Bitstream.
3. Vivado will likely ask if it should run Synthesis and Implementation first. Click Yes.
4. This process will take a few minutes. You can watch the progress bar in the top right corner. When it finishes, a "Bitstream Generation Completed" window will pop up.

3. Connect the Basys 3 Board

1. Plug your Basys 3 board into your computer using a micro-USB cable.
2. Make sure the jumper block next to the power switch is set to USB.
3. Flip the power switch on the board to the ON position. The red power LED on the board should light up.

4. Program the FPGA

1. In the "Bitstream Generation Completed" popup window, select Open Hardware Manager and click OK. (If you dismissed the popup, you can just click "Open Hardware Manager" at the bottom left of Vivado).
2. At the top of the Hardware Manager window, click the green banner that says Open target, then select Auto Connect.

TECHNICAL DOCUMENTATION of Anviksha 8 – Bit CPU

3. Vivado will scan your USB ports and find the Basys 3 board (it will show up under "Hardware" as xc7a35t).
4. Right-click on xc7a35t and select Program Device.
5. A window will pop up showing the path to your .bit file. Leave everything as default and click Program.
6. A green progress bar will flash across the screen. When it finishes, the orange "DONE" LED on your Basys 3 board will light up. Your CPU is now physically wired into the silicon!

5. Execute the Fibonacci Benchmark Live

Now that the FPGA is programmed, it behaves exactly like the testbench you simulated! Here is your physical launch sequence:

1. Push SW1 UP: This activates prog_mode_sw. The fast clock takes over, and your hardware Program Loader instantly writes your Fibonacci machine code into the SRAM.
2. Push SW0 UP, then DOWN: This triggers reset_sw. It clears the Program Counter and gets the CPU ready at Address 00.
3. Push SW1 DOWN: This turns off Program Mode. The CPU wakes up and connects to the human-visible clock (clk_div[24]).

Relevant Images:

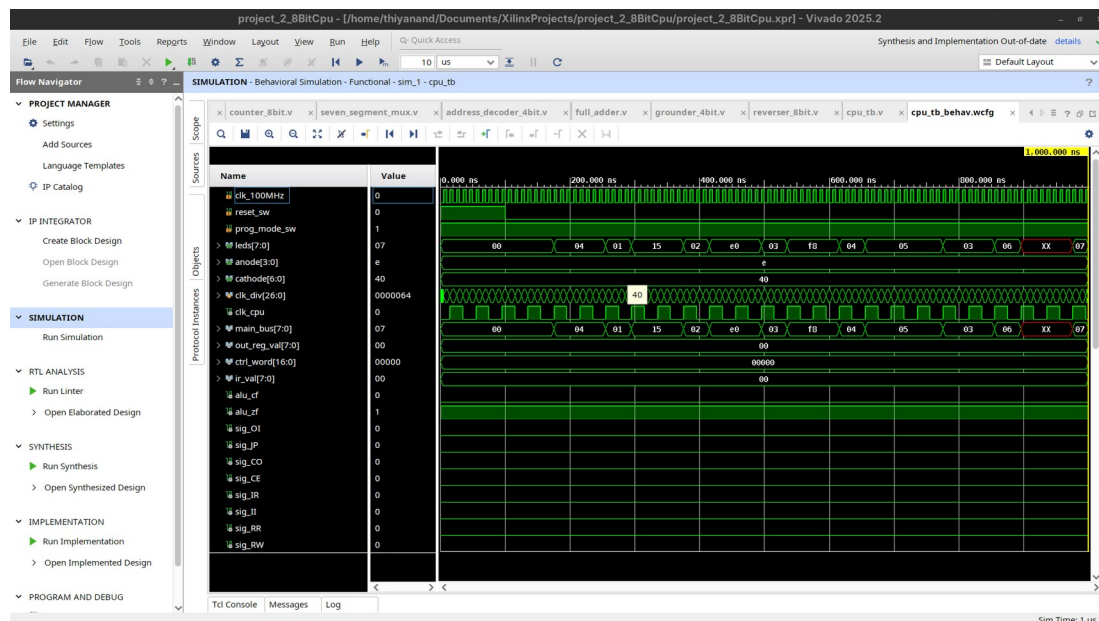


Figure 1: Behavioral Simulation of Adding Two Variables, showing the loading of data onto the 8-bit main bus.

TECHNICAL DOCUMENTATION of Anviksha 8 – Bit CPU

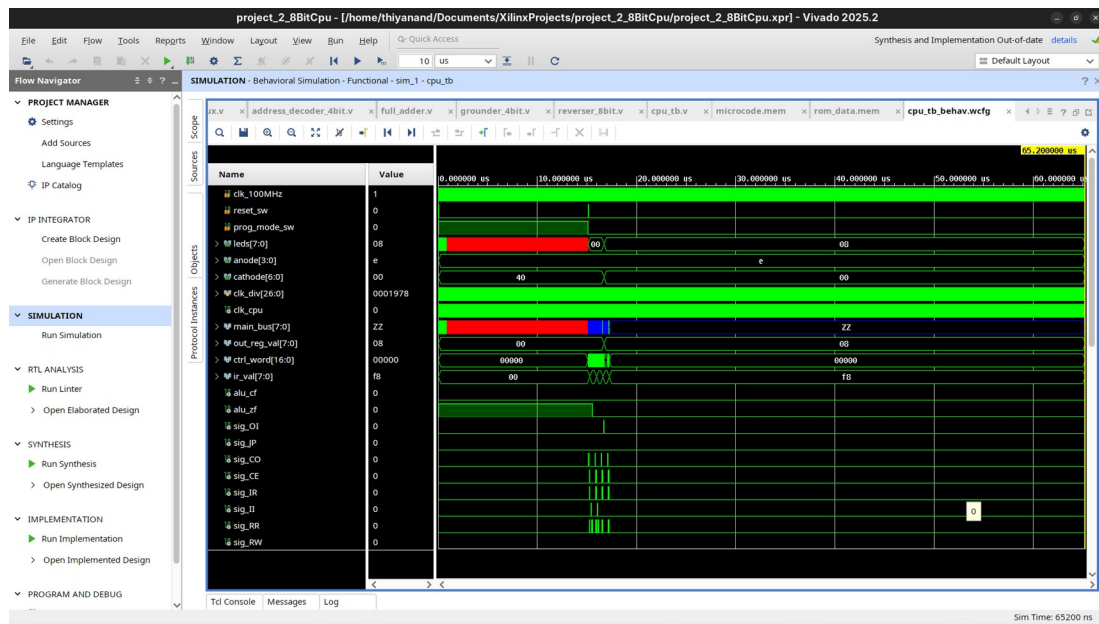


Figure 2: Behavioral Simulation of Adding Two Variables, showing the output execution of the assembly-level code.

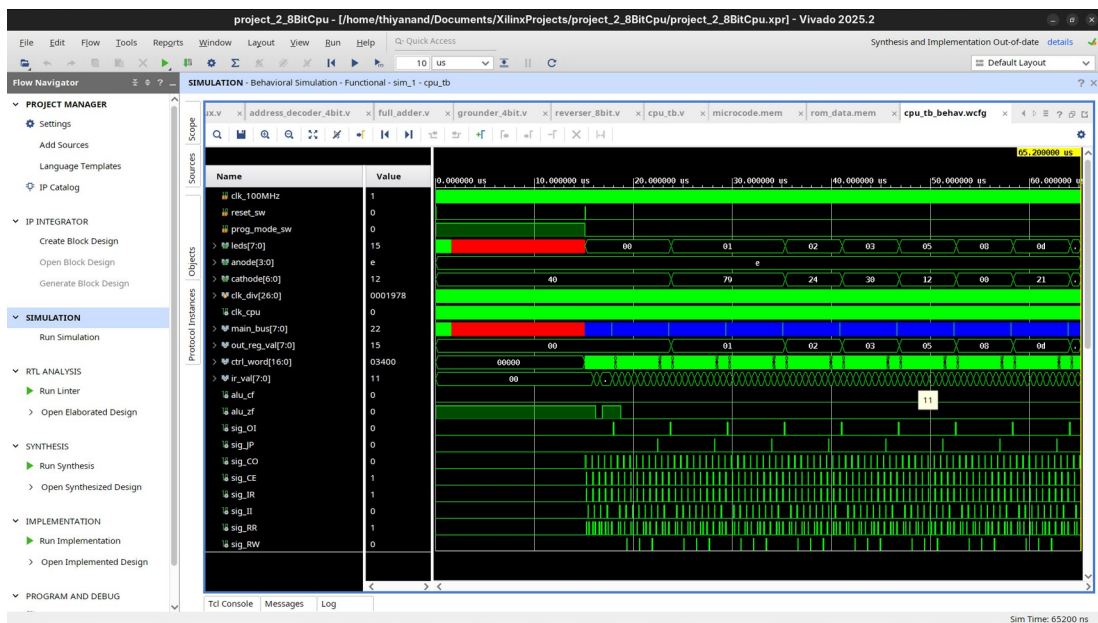


Figure 3: Behavioural Simulation of the Fibonacci Sequence.

TECHNICAL DOCUMENTATION of Anviksha 8 – Bit CPU

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	7.567 ns	Worst Hold Slack (WHS):	0.252 ns	Worst Pulse Width Slack (WPWS):	4.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	45	Total Number of Endpoints:	45	Total Number of Endpoints:	46

All user specified timing constraints are met.

Figure 4: Timing Report

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 0.081 W

Design Power Budget: Not Specified

Process: typical

Power Budget Margin: N/A

Junction Temperature: 25.4°C

Thermal Margin: 59.6°C (11.9 W)

Ambient Temperature: 25.0 °C

Effective θJA: 5.0°C/W

Power supplied to off-chip devices: 0 W

Confidence level: Low

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

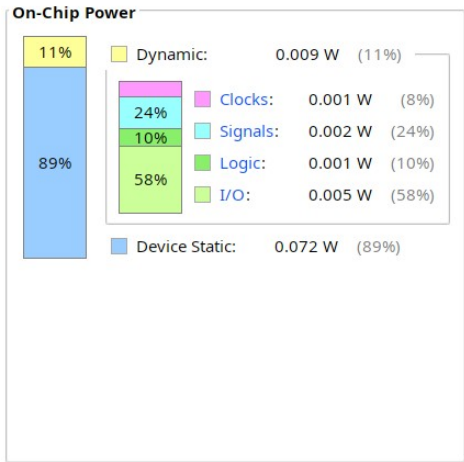


Figure 5: Power Report

